

Before this course, I had no formal understanding of distributed systems. My experience with backend development was limited to implementing REST APIs and deploying them using managed platforms such as Vercel. I viewed deployment primarily as a configuration step that followed coding, without deeply considering scalability, concurrency, fault tolerance, or performance under load. Throughout this course, my perspective shifted significantly. I developed a clear understanding of what constitutes a distributed system, how multiple components interact across isolated environments, and how such systems must be designed to handle scale and partial failure. Through hands-on assignments involving Docker, Amazon ECS, and Application Load Balancers, I moved beyond writing standalone services and began thinking in terms of system architecture, resource management, and observable behavior under real-world conditions.

Application Load Balancers and Load Testing

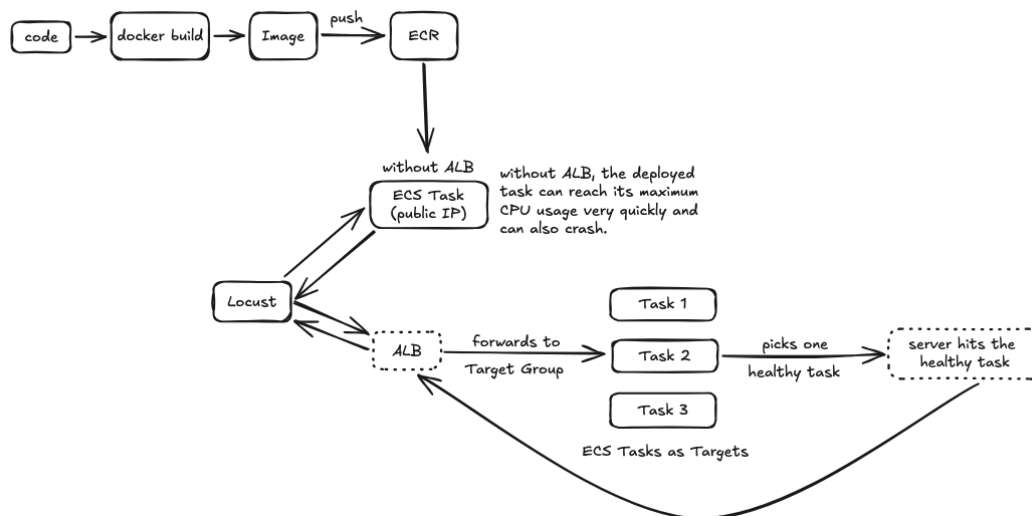
What I enjoyed most: Implementing horizontal scaling using an Application Load Balancer (ALB) and validating system behavior through Locust-based load testing.

Why it mattered: It turned scalability from a lecture concept into something I could measure and observe in a real deployment.

Scale-out insight: Running multiple ECS tasks behind an ALB demonstrated how traffic can be distributed across replicas, improving resilience and reducing per-task load, compared to scaling up a single instance.

Performance metrics learned: Load testing helped me distinguish throughput (RPS) from latency, including higher percentiles (P95/P99), which better capture user experience under saturation.

Bottleneck thinking: Optimizing the load generator reduced observed response times, but overall throughput remained bounded by backend capacity, reinforcing the need to locate the true system bottleneck.



Containers, Docker, and the ECS Execution Model

- Before this course, I did not understand the distinction between a Docker image and a container. I learned that an image is an immutable blueprint containing code and dependencies, while a container is a running instance of that image.
- Building images, pushing them to Amazon ECR, and deploying them as ECS tasks clarified how cloud systems achieve reproducible and isolated execution environments.
- I understood that an ECS task represents an orchestrated container instance, which makes horizontal replication straightforward and programmatic.
- This experience shifted my perspective from deploying code to deploying scalable, replaceable execution units that enable elastic, distributed architectures.

Modularity and System Decomposition (Parnas)

- One theoretical topic that stood out to me was Parnas' argument that systems should be decomposed based on information hiding rather than processing steps. This challenged the intuitive but fragile approach of splitting systems by workflow stages.
- I began to understand that well-designed services encapsulate volatile decisions internally and expose only stable interfaces.
- This idea directly relates to microservices architecture, where independently deployable services must remain loosely coupled and cohesive.
- The assignment work with containers and ECS made this principle concrete: each service replica encapsulated its dependencies and could scale independently without affecting other components.

Distributed Transactions and Atomicity (2PC)

Learning about Two-Phase Commit (2PC) introduced me to the challenges of maintaining atomicity across multiple distributed components. Unlike single-node transactions, distributed commits require coordination between independent participants.

Understanding 2PC helped me appreciate why many modern distributed architectures avoid tightly coupled distributed transactions and instead favor eventual consistency and decoupled services.

Conclusion

Overall, this course reshaped my understanding of backend systems from isolated API development to distributed system design. Through hands-on assignments and foundational readings, I developed both practical and conceptual clarity regarding scalability, modularity, and infrastructure orchestration. Implementing load-balanced ECS services while understanding the theoretical principles of decomposition and abstraction allowed me to connect system behavior with architectural decisions. The combination of theory and measurable experimentation has significantly influenced how I now approach system design and cloud-based application development.